SHOPSPHERE

DevOps Project Handover & Implementation Record

Terraform • AWS EC2 • Docker Compose • Amazon ECR • Jenkins CI/CD • AWS Systems Manager

Environment: Windows Jenkins controller + Ubuntu EC2
AWS region: ap-southeast-2 (Sydney)
Record date: 10 November 2026

Scope: This document records the implementation and troubleshooting completed up to the point where the Jenkins pipeline, including SSM-based EC2 deployment, was reported as **Finished: SUCCESS**. Confirmed work is separated from outstanding tasks.

Security: no AWS keys, passwords, JWT values, or private keys are included.

1. Project overview

ShopSphere is a containerized application composed of three backend microservices, a frontend, and an Nginx reverse proxy. Work covered Terraform provisioning, Ubuntu EC2 preparation, Docker deployment, ECR image publishing, Jenkins CI, and SSM-based continuous deployment.

Component	Service name	Port / purpose
User backend	user-service	3001
Product backend	product-service	3002
Order backend	order-service	3003
Frontend	frontend	Web UI container
Reverse proxy	nginx	Public HTTP port 80

Architecture

GitHub → Jenkins (Windows) → test and build four Docker images → Amazon ECR → AWS SSM Run Command → Ubuntu EC2 → Docker Compose services behind Nginx.

Terraform provisions the EC2 host, security group, instance role/profile and related infrastructure. EC2 uses its instance role for ECR pulls and SSM. Jenkins uses its configured AWS credential for ECR pushes and SSM command submission.

Setting	Recorded value
AWS account	507941515612
AWS region	ap-southeast-2 (Sydney)
EC2 instance	i-0af087445cc5f5878
EC2 public IP	13.239.24.127
EC2 public DNS	ec2-13-239-24-127.ap-southeast-2.compute.amazonaws.com
EC2 app directory	/opt/shopsphere
Local project	C:\Users\Shreyas\Microservice-application
Jenkins job / URL	E-Commerce-CI-CD / http://localhost:8080
ECR registry	507941515612.dkr.ecr.ap-southeast-2.amazonaws.com

2. Terraform and AWS infrastructure

Terraform was installed locally (reported version 1.17.1) and initialized with AWS provider 6.72.0. The dedicated AWS CLI profile was named terraform and the region was ap-southeast-2.

```
# Windows PowerShell
$env:AWS_PROFILE = "terraform"
$env:AWS_DEFAULT_REGION = "ap-southeast-2"
terraform init
terraform validate
terraform plan -out=tfplan
terraform apply tfplan
```

The reported plan was 6 to add, 0 to change, 0 to destroy. Apply completed successfully.

Terraform file	Purpose
providers.tf	AWS provider configuration
variables.tf	Project, environment, region and instance variables
main.tf	EC2 instance and AMI/subnet data
iam.tf	EC2 IAM role and instance profile
security-groups.tf	Project security group
ecr.tf	Data lookups for existing ECR repositories
outputs.tf	EC2 and ECR outputs

• Existing default VPC: vpc-084fd5852925f594d (172.31.0.0/16).

• Recorded subnet: subnet-0ce7e4a3b96ba1e4a in ap-southeast-2a.

• EC2: Ubuntu 24.04, t3.small, 20 GB encrypted gp3 root volume, public IP enabled.

• Security group: shopsphere-terraform-sg; inbound TCP 80 from 0.0.0.0/0; outbound allowed. No public SSH ingress was added to this project SG.

• EC2 role: ShopSphere-EC2-Role; instance profile: ShopSphere-EC2-Profile.

• Role policies: AmazonEC2ContainerRegistryReadOnly and AmazonSSMManagedInstanceCore.

• Four existing ECR repositories were reused: shopsphere-user, shopsphere-product, shopsphere-order, shopsphere-frontend.

State hygiene: ignore terraform/terraform.tfstate, terraform/terraform.tfstate.backup and terraform/.terraform/ in Git. Keep .terraform.lock.hcl committed. Do not commit state files.

3. EC2 server preparation and SSM

Access method

Windows AWS CLI and Session Manager plugin were installed. Interactive access was established through SSM rather than SSH/private key distribution.

```
# Windows PowerShell
aws ssm start-session --target i-0af087445cc5f5878 --profile terraform --region ap-southeast-2
```

On EC2, aws sts get-caller-identity returned an assumed-role identity for ShopSphere-EC2-Role, confirming temporary instance-role credentials.

Software installed on Ubuntu

• Docker Engine 29.9.3 and Docker Compose 5.7.0; docker run hello-world succeeded.

• AWS CLI v2.39.7.

• Amazon SSM Agent snap 3.3.4793.0; service was active/running.

• SSM session shell user was ssm-user; host shown as ip-172-31-7-251.

A point-in-time disk check showed a 19 GB root filesystem, around 4 GB used and 15 GB free (22% used).

IAM permissions

EC2 instance role had AmazonSSMManagedInstanceCore. Jenkins IAM user was jenkins-shopsphere. An inline policy named ShopSphere-Jenkins-SSM-Deploy was added/prepared with these actions:

```
ssm:SendCommand
ssm:GetCommandInvocation
ssm:ListCommandInvocations
ssm:DescribeInstanceInformation
```

The policy used Resource: "*" as a learning-stage permission set. It should be narrowed for production, especially SendCommand to approved SSM documents and the intended instance where supported.

Jenkins credential ID: aws-ecr-jenkins. It stores an AWS access key ID and secret access key as a Jenkins username/password credential. Secret values are deliberately omitted from this handover.

Security practices

• No EC2 private key is needed for the SSM deployment path.

• EC2 obtains ECR pull access through its instance role.

• Jenkins obtains ECR push and SSM permissions through its Jenkins credential/IAM user.

• Application .env on EC2 contains a JWT secret; it was kept separate from image-tag.env. Never print or commit .env.

• Public access is HTTP on port 80; HTTPS was not configured in the work completed.

4. Docker Compose deployment on EC2

Deployment directory: /opt/shopsphere. It contains docker-compose.yml, image-tag.env, .env and nginx/nginx.conf.

Nginx routing

Incoming path	Upstream
/	frontend:80
/api/users/	user-service:3001
/api/products/	product-service:3002
/api/orders/	order-service:3003

Nginx publishes port 80 and proxies requests to the frontend and backend services. The Nginx config was mounted read-only at /etc/nginx/nginx.conf.

Initial release

Four ECR images tagged v1.0.0 were pulled and launched using Docker Compose. The public application was verified by the user from a laptop and mobile devices at http://13.239.24.127.

```
# Ubuntu EC2
cd /opt/shopsphere
sudo docker compose config -q
sudo docker compose up -d
sudo docker compose ps
curl -I http://localhost/
```

Backend healthchecks use Node.js HTTP requests to /health. The stack includes user-service, product-service, order-service, frontend and nginx.

Persistence warning: earlier project notes mentioned /app/data for user/order services, but the described EC2 Compose file did not include persistent volumes. Verify whether important data is written to container filesystems before future redeployments; add volumes or an external datastore if needed.

5. Windows laptop and Jenkins setup

• Local project directory: C:\Users\Shreyas\Microservice-application.

• Jenkins installed on Windows; UI at http://localhost:8080.

• Jenkins job: E-Commerce-CI-CD.

• AWS CLI v2.35.1 reported on Windows.

• Docker Desktop/Engine used by Jenkins for Docker builds.

• Jenkins AWS credential ID: aws-ecr-jenkins.

Relevant repository layout

```
Microservice-application/
■■■ Jenkinsfile
■■■ docker-compose.yml
■■■ .env.example
■■■ services/
■   ■■■ user-service/Dockerfile
■   ■■■ product-service/Dockerfile
■   ■■■ order-service/Dockerfile
■■■ frontend/Dockerfile
■■■ nginx/nginx.conf
■■■ scripts/deploy-ssm.ps1
■■■ terraform/
    ■■■ providers.tf
    ■■■ variables.tf
    ■■■ main.tf
    ■■■ iam.tf
    ■■■ security-groups.tf
    ■■■ ecr.tf
    ■■■ outputs.tf
```

This is a summary of relevant paths discussed, not a complete listing of every application source file.

Credential binding

The Jenkinsfile uses withCredentials with usernamePassword(credentialsId: 'aws-ecr-jenkins', usernameVariable: 'AWS_ACCESS_KEY_ID', passwordVariable: 'AWS_SECRET_ACCESS_KEY'). Jenkins masks supported secret patterns in logs. Never echo credentials or dump the environment.

The job successfully checked out source through SCM. A GitHub webhook/polling trigger was an assignment requirement, but the exact active trigger configuration was not verified in the conversation.

6. Jenkins CI and ECR publishing

The original pipeline used a fixed v1.0.0 tag and expected short local image names. That did not match the actual fully qualified ECR image names. The pipeline was updated to use a unique build tag and explicit Docker build commands.

Pipeline stage	Action
Source Checkout	checkout scm
Set Release Version	IMAGE_TAG = build-${BUILD_NUMBER}
Prepare Environment	Copy .env.example to .env only if .env is absent
Configuration Validation	docker compose config -q
Application Tests	npm ci and npm test --if-present for user, product and order services
Build Docker Images	Build four images using their Dockerfiles and full ECR names
Verify Docker Images	docker image inspect on all four fully qualified image:tag values
Authenticate and Push to ECR	ECR login and push four images
Deploy to EC2 through SSM	Run scripts/deploy-ssm.ps1 with the current IMAGE_TAG

Explicit image build commands

```
docker build -t %ECR_REGISTRY%/shopsphere-user:%IMAGE_TAG% -f services/user-service/Dockerfile services/user-service
docker build -t %ECR_REGISTRY%/shopsphere-product:%IMAGE_TAG% -f services/product-service/Dockerfile services/product-service
docker build -t %ECR_REGISTRY%/shopsphere-order:%IMAGE_TAG% -f services/order-service/Dockerfile services/order-service
docker build -t %ECR_REGISTRY%/shopsphere-frontend:%IMAGE_TAG% -f frontend/Dockerfile frontend
```

ECR publishing

```
docker push %ECR_REGISTRY%/shopsphere-user:%IMAGE_TAG%
docker push %ECR_REGISTRY%/shopsphere-product:%IMAGE_TAG%
docker push %ECR_REGISTRY%/shopsphere-order:%IMAGE_TAG%
docker push %ECR_REGISTRY%/shopsphere-frontend:%IMAGE_TAG%
```

Jenkins ECR login returned Login Succeeded. A later CI run reported Finished: SUCCESS after the image naming/build changes. A subsequent run including the SSM deployment stage also reported Finished: SUCCESS. The exact final successful build number was not supplied.

7. Continuous deployment through SSM

SSM validation and troubleshooting

• SSM Agent was active; snap version 3.3.4793.0.

• An early multiline SSM command failed with _script.sh not found. Basic SSM execution was later verified successfully.

• An echo test returned SSM_DEPLOYMENT_WORKS.

• A multi-command test ran pwd and docker compose ps successfully in /opt/shopsphere. The output showed all five containers up and backend services healthy.

• PowerShell inline JSON quoting caused ParamValidation errors. The working approach wrote JSON to a temporary file using UTF-8 without BOM and sent it using --cli-input-json file://... .

EC2 Compose dynamic tags

Inspection showed the EC2 Compose file initially used fixed v1.0.0 tags and image-tag.env contained IMAGE_TAG=v1.0.0. The four application image references were changed to support IMAGE_TAG interpolation:

```
image: 507941515612.dkr.ecr.ap-southeast-2.amazonaws.com/shopsphere-user:${IMAGE_TAG:-v1.0.0}
image: 507941515612.dkr.ecr.ap-southeast-2.amazonaws.com/shopsphere-product:${IMAGE_TAG:-v1.0.0}
image: 507941515612.dkr.ecr.ap-southeast-2.amazonaws.com/shopsphere-order:${IMAGE_TAG:-v1.0.0}
image: 507941515612.dkr.ecr.ap-southeast-2.amazonaws.com/shopsphere-frontend:${IMAGE_TAG:-v1.0.0}
```

Nginx remained nginx:alpine. Compose validation using --env-file image-tag.env succeeded. The tag file initially contained IMAGE_TAG=v1.0.0. The app .env was not overwritten.

Local PowerShell deployment script

A script was created at scripts/deploy-ssm.ps1. It validates that the tag follows build-N, builds an AWS-RunShellScript payload, writes JSON without BOM, submits the command, polls get-command-invocation, displays output, and fails the script if SSM reports a failed status.

Remote deployment sequence

```
cd /opt/shopsphere
# Read previous release tag
PREVIOUS_TAG=$(sed -n 's/^IMAGE_TAG=//p' image-tag.env | head -n 1)

# Set new release tag
echo "IMAGE_TAG=<JENKINS_BUILD_TAG>" > image-tag.env

# Authenticate using the EC2 instance role
aws ecr get-login-password --region ap-southeast-2 |
  docker login --username AWS --password-stdin 507941515612.dkr.ecr.ap-southeast-2.amazonaws.com

# Pull and apply
docker compose --env-file image-tag.env pull
docker compose --env-file image-tag.env up -d --remove-orphans

# Basic HTTP availability check
curl -fsS http://localhost/ > /dev/null
```

The Jenkinsfile added a Deploy to EC2 through SSM stage that invokes the PowerShell script with %IMAGE_TAG%. It uses the existing aws-ecr-jenkins credential binding.

Rollback is not implemented yet. The script reads the previous tag into a variable, but the shared version does not restore that tag on failure. The current check is a basic HTTP homepage check. Automatic rollback and deeper service-level health checks remain outstanding.

The user reported Finished: SUCCESS after adding this SSM deployment stage. An independent inspection of the exact running tag after that final run had not yet been shared.

8. Troubleshooting record

Problem	What happened / resolution
SSM _script.sh not found	Initial multiline SSM request failed. SSM Agent and disk were healthy. Basic echo and JSON-file payload methods later worked.
Invalid SSM JSON	PowerShell quoting and UTF-8 BOM issues. Resolved by generating JSON and writing UTF-8 without BOM, then --cli-input-json file://... .

Problem	What happened / resolution
sed syntax error	First replacement delimiter conflicted with slash characters. Switched to # delimiter; four Compose image references updated.
Docker named pipe unavailable	Jenkins initially could not connect to npipe:////./pipe/docker_engine. Docker connection was restored; subsequent pipeline progressed.
Jenkins image inspect failed	Expected ECR image:build-N tags did not exist. Replaced Compose-dependent build with explicit docker build commands using full ECR names.
ECR push tag missing	Build-22 tag was absent locally. Deterministic image build/tag stage fixed the mismatch; later pipeline succeeded.
ECR authentication	aws ecr get-login-password piped to docker login; Login Succeeded.

9. Current status and remaining work

Area	Status at handover
Terraform provisioning	Completed; apply created 6 resources.
EC2 and server setup	Completed; Ubuntu, Docker, Compose, AWS CLI, SSM Agent.
IAM / SSM managed instance	Completed; role credentials and SSM Run Command verified.
Application on EC2	Deployed and previously verified publicly over HTTP; initial tag v1.0.0.
Four ECR repositories	Reused; successful image publishing confirmed.
Jenkins CI	Successful run confirmed.
Jenkins SSM CD stage	User reported Finished: SUCCESS.
Post-deployment running tag verification	Pending latest confirmation.
Automatic rollback	Pending; not in the script version shared.
Prometheus / Node Exporter / Grafana	Not yet installed/configured.
Failure simulation / recovery evidence	Pending.
README, architecture diagram, screenshots	Pending final evidence pack.

Recommended next sequence

• 1. Verify the latest deployed version on EC2: cat image-tag.env; docker compose --env-file image-tag.env ps; curl -I http://localhost/.

• 2. Add real rollback: preserve old tag; on failed pull/start/health checks, restore old tag, pull previous images and restart old stack; report rollback outcome.

• 3. Add service-level checks for the three backend /health endpoints, not only homepage HTTP.

• 4. Install Prometheus and Node Exporter; configure scrape targets.

• 5. Install Grafana and build dashboards for CPU, memory, disk, node metrics and application/container availability.

• 6. Run controlled failure simulation and capture recovery evidence.

• 7. Complete README, architecture diagram, deployment guide, rollback/troubleshooting notes and screenshots.

• 8. Review AWS costs; only destroy infrastructure after all evidence is captured.

Cost note: t3.small EC2, 20 GB gp3 EBS, public IPv4, ECR storage and data transfer may incur charges. AWS credits were mentioned by the user but the remaining balance was not verified. Check Billing/Cost Explorer. Do not run terraform destroy before evidence capture.

10. Operational command reference

Windows PowerShell

```
# Project
cd C:\Users\Shreyas\Microservice-application

# Docker connectivity
docker version
docker info
docker ps

# SSM interactive session
aws ssm start-session --target i-0af087445cc5f5878 --profile terraform --region ap-southeast-2

# Run deployment script intentionally
powershell.exe -NoProfile -ExecutionPolicy Bypass -File .\scripts\deploy-ssm.ps1 -ImageTag build-<N>
```

Ubuntu EC2 via SSM

```
cd /opt/shopsphere
sudo docker compose ps
sudo docker compose logs --tail=100
cat image-tag.env
curl -I http://localhost/
df -h
sudo snap services amazon-ssm-agent
```

Jenkins

• Jenkins URL: http://localhost:8080

• Job: E-Commerce-CI-CD

• Credential ID: aws-ecr-jenkins

• Check Console Output for release tag, tests, build, ECR pushes and SSM deployment result.

• Never print AWS_ACCESS_KEY_ID, AWS_SECRET_ACCESS_KEY, JWT secret or .env contents.

Release readiness checklist

• Confirm intended Git commit is checked out.

• Confirm Dockerfiles and build contexts.

• Confirm four ECR repositories exist in ap-southeast-2.

• Confirm EC2 is SSM Online and instance role can pull ECR images.

• Confirm EC2 Compose has IMAGE_TAG interpolation for all four application images.

• Confirm image-tag.env represents the currently deployed release.

• After each deployment, verify service health and HTTP responses; a green Jenkins result alone is not proof of application correctness.

Handover summary: The project has a Terraform-provisioned EC2 host, deployed Docker Compose application, four ECR repositories, and a Jenkins pipeline that has reported success through SSM-based deployment. Next engineering work: verify deployed release, implement rollback, add monitoring/alerting, demonstrate failure recovery, and finish assignment evidence.

End of handover document • ShopSphere DevOps project • 10 November 2026